

Detecção de Armadilhas de Trânsito em Malhas Viárias por Componentes Fortemente Conexas: Fase I do Projeto Integrador 3B

Caio Freitas Leandro (2422130050), Maurício da Paixão (2422130010)

Rafael Amorim (2422130043), Pedro Oliveira (2412130172)

Projeto Integrador 3B — Disciplina de Teoria dos Grafos

Resumo. Este trabalho apresenta a Fase I de um projeto de análise de transportes urbanos que modela a malha viária como um grafo direcionado e busca responder se existem “armadilhas de trânsito”: regiões onde um veículo entra e, pelo sentido das vias, não consegue mais sair. Define-se armadilha como uma componente fortemente conexa (CFC) terminal no grafo de condensação, e a detecção é feita em C com o algoritmo de Tarjan sobre lista de adjacência, comparada a uma implementação com matriz de adjacência. Em grafos sintéticos de até 10.000 vértices e 40.000 arestas, o Tarjan com lista levou 2,4 ms no maior caso, e a matriz exigiria cerca de 381 MiB contra 0,7 MiB da lista. Em um recorte do centro de São Paulo extraído do OpenStreetMap (6.039 vértices, 8.154 arestas), foram encontradas 979 CFCs, uma CFC gigante com 4.786 vértices (79,3%) e 60 CFCs terminais que reúnem 228 vértices (3,8%); 50 delas são alcançáveis a partir da CFC gigante. A análise experimental também mostra que o gargalo do pipeline não é o Tarjan, e sim a contagem quadrática de arestas duplicadas na leitura.

Abstract. *This paper presents Phase I of an urban transportation project that models a road network as a directed graph and asks whether “traffic traps” exist: regions that a vehicle can enter but, because of one-way streets, cannot leave. A traffic trap is defined as a terminal strongly connected component (SCC) of the condensation graph, and detection is implemented in C using Tarjan’s algorithm over adjacency lists, compared with an adjacency-matrix implementation. On synthetic graphs of up to 10,000 vertices and 40,000 edges, Tarjan on lists took 2.4 ms in the largest case, while the matrix would require about 381 MiB versus 0.7 MiB for the list. On an OpenStreetMap extract of downtown São Paulo (6,039 vertices, 8,154 edges), we found 979 SCCs, a giant SCC with 4,786 vertices (79.3%) and 60 terminal SCCs holding 228 vertices (3.8%); 50 of them are reachable from the giant SCC. The experiments also show that the pipeline bottleneck is not Tarjan’s algorithm but the quadratic duplicate-edge counting performed while reading the input.*

1. Introdução

Em uma cidade, o sentido das vias importa tanto quanto a sua existência. Ruas de mão única e arteriais com acesso restrito podem formar regiões em que o motorista consegue entrar, mas não consegue voltar ao ponto de origem sem uma manobra proibida. Para o planejamento urbano e para sistemas de navegação, saber onde essas regiões estão é uma pergunta de conectividade, e a Teoria dos Grafos oferece ferramentas diretas para respondê-la.

Este artigo descreve a Fase I do Projeto Integrador 3B, cujo objetivo é aplicar conceitos de grafos à análise de transportes urbanos. A Fase I trata de topologia e conectividade (sem pesos nas arestas); a Fase II, de rotas e otimização. A pergunta de negócio que guia o trabalho é: *existem armadilhas de trânsito, isto é, bairros onde um carro entra mas, por conta do sentido das vias, não consegue mais sair e voltar ao ponto de origem?*

As contribuições são: (i) um modelo de grafo direcionado para a malha viária e uma definição operacional de armadilha de trânsito baseada em componentes fortemente conexas; (ii) um pipeline em C11, com lista e matriz de adjacência, algoritmo de Tarjan, detecção de armadilhas, medição de tempo e registro de resultados em CSV; (iii) uma avaliação experimental em grafos sintéticos (100 a 10.000 vértices) e em um recorte real do centro de São Paulo; e (iv) uma análise crítica dos resultados, incluindo limitações da implementação e das medições. O restante do texto está organizado assim: a Seção 2 revisa a fundamentação; a Seção 3 apresenta o modelo e a definição de armadilha; a Seção 4 descreve a implementação; a Seção 5 apresenta os dados; a Seção 6 discute os resultados; a Seção 7 trata das limitações; e a Seção 8 conclui.

2. Fundamentação teórica

2.1. Grafos direcionados e representações

Um grafo direcionado $G = (V, E)$ é formado por um conjunto de vértices V e um conjunto de arestas orientadas E , cada uma um par ordenado (u, v) (Cormen et al., 2009). Duas representações são clássicas. Na **lista de adjacência**, cada vértice guarda a lista de seus sucessores; a memória é $O(V + E)$ e percorrer os vizinhos de u custa $O(\text{grau de saída de } u)$. Na **matriz de adjacência**, uma matriz $V \times V$ indica a existência de cada aresta; a consulta de uma aresta é $O(1)$, mas a memória é $O(V^2)$ e percorrer os vizinhos de u custa $O(V)$. Malhas viárias são esparsas, o que favorece a lista.

2.2. Componentes fortemente conexas

Uma componente fortemente conexa (CFC) é um conjunto maximal C de vértices tal que, para todo par u, v em C , existem caminhos de u até v e de v até u . Contrair cada CFC em um único vértice produz o *grafo de condensação*, que é sempre um grafo acíclico dirigido (DAG). Uma CFC é **terminal** (ou sumidouro) quando nenhuma aresta sai dela para outra CFC: quem chega a uma CFC terminal nunca mais deixa o seu conjunto de vértices.

2.3. O algoritmo de Tarjan

O algoritmo de Tarjan (1972) encontra todas as CFCs com uma única busca em profundidade. Cada vértice recebe um índice de descoberta e um valor *low-link* (o menor índice alcançável por ele, seguindo arestas da árvore de busca e no máximo uma aresta para um vértice ainda na pilha). Os vértices são empilhados à medida que são descobertos; quando o *low-link* de v é igual ao seu índice, v é a raiz de uma CFC e a pilha é desempilhada até v . Cada vértice e cada aresta são visitados uma vez, e a complexidade é $O(V + E)$. A alternativa clássica, o algoritmo de Kosaraju–Sharir (Sharir, 1981), faz duas buscas e exige o grafo transposto; o projeto adotou Tarjan por usar uma única DFS e dispensar a transposição.

3. Modelagem do problema

3.1. A malha viária como grafo

A malha é modelada como $G = (V, E)$. Cada vértice é um ponto da malha, identificado por um inteiro de 0 a $n-1$, e cada aresta (u, v) é um trecho de rua que pode ser percorrido de u para v . Uma via de mão única gera uma aresta; uma via de mão dupla gera duas, (u, v) e (v, u) . Na Fase I o grafo não é ponderado; o tempo de trânsito como peso entra na Fase II.

3.2. Definição de armadilha de trânsito

Seja C uma CFC de G . O trabalho adota a seguinte definição operacional: **uma armadilha de trânsito é uma CFC terminal no grafo de condensação**, ou seja, uma CFC da qual não sai nenhuma aresta para outra CFC. Um veículo que entra em C permanece em C para sempre; dentro de C , porém, há sempre como voltar (se $|C| > 1$ ou se há auto-laço). Além do caráter terminal, cada CFC é classificada em: (a) tamanho 1 sem auto-laço; (b) tamanho 1 com auto-laço; (c) tamanho maior que 1. Uma CFC terminal de tamanho 1 sem auto-laço é um vértice sem arestas de saída, ou seja, um beco sem saída.

Duas observações ajudam a interpretar a definição. Primeiro, ela é conservadora: toda aresta entre CFCs distintas é irreversível (não há caminho de volta), mas só as CFCs terminais impedem qualquer saída. Segundo, se o grafo inteiro é fortemente conexo, ele tem uma única CFC, que é trivialmente terminal e não representa armadilha alguma, pois não há nada fora dela de onde se possa entrar. Por isso, ao interpretar os números, convém distinguir as CFCs terminais que de fato podem ser alcançadas a partir de outras regiões (Seção 6.4).

4. Implementação

4.1. Visão geral

O sistema foi escrito em C11 (ISO/IEC 9899:2011), com identificadores e comentários em português, e organizado em cabeçalhos (*cabeçalhos/*), implementações (*fontes/*), testes, dados, documentação, resultados e scripts. A Figura 1 resume o fluxo: um extrato do OpenStreetMap é convertido em lista de arestas, lido e validado, montado em lista ou matriz de adjacência, processado pelo Tarjan e pela análise de armadilhas, e os resultados são registrados em CSV para geração de gráficos. O código está em github.com/AmorimRafa/Projeto-Integrador-3B.

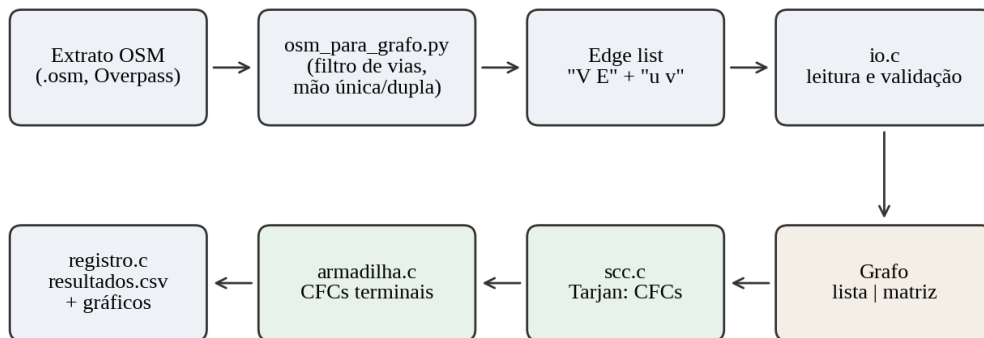


Figura 1. Fluxo de processamento do projeto, do extrato OSM ao registro dos resultados.

4.2. Entrada, leitura e validação

O formato de entrada é uma lista de arestas em texto: a primeira linha traz “V E” e cada linha seguinte traz uma aresta “u v”. Linhas em branco e linhas iniciadas por “#” são ignoradas. O módulo *io.c* lê o cabeçalho, carrega as arestas em vetores dinâmicos, valida que todos os extremos estão em $[0, V)$ e conta auto-laços e arestas duplicadas (pares de origem e destino repetidos).

4.3. Estruturas de dados

A lista de adjacência usa um vetor de ponteiros para nós encadeados (NoViz, com o vértice de destino e o próximo nó); a inserção é feita na cabeça da lista, em $O(1)$. A matriz é um vetor de V linhas alocadas com `calloc`, com $m[u][v] = 1$ quando a aresta existe; por ser uma matriz de inteiros de 4 bytes, ela colapsa arestas repetidas em uma só. Um contêiner Grafo despacha as operações conforme a representação escolhida. Ambos os módulos estimam a própria memória: a lista usa $16 + 8V + 16E$ bytes e a matriz usa $16 + 8V + 4V^2$ bytes, assumindo ponteiros de 8 bytes (`sizeof(NoViz) = 16` com alinhamento). Para o grafo de teste ($V = 5$, $E = 6$) esses cálculos dão 152 e 156 bytes, os mesmos valores informados pelo programa.

4.4. Tarjan e análise de armadilhas

O Tarjan (`SCC.c`) é recursivo e mantém índices, low-links, uma pilha e uma marca de presença na pilha. Sua saída é o vetor `comp[v]` (CFC de cada vértice) e o vetor `tam[c]` (tamanho de cada CFC). A análise de armadilhas (`armadilha.c`) segue o procedimento do Algoritmo 1.

Algoritmo 1: armadilhas a partir das CFCs

```
1 para cada CFC c: terminal[c] <- 1
2 para cada aresta (u, v) do grafo:
3     se comp[u] != comp[v]: terminal[comp[u]] <- 0
4 para cada CFC c:
5     se tam[c] > 1: conta "tamanho > 1"
6     senao: localiza o único vértice u de c
7         conta "com laço" se (u, u) existe, "sem laço" caso contrário
8 n_terminais <- número de c com terminal[c] = 1
```

A varredura de arestas (linhas 2 e 3) custa $O(V + E)$. A localização do vértice de cada CFC unitária (linha 6) é feita por busca linear no vetor `comp`; no pior caso, o custo total dessa etapa é $O(V \cdot C)$, em que C é o número de CFCs, o que vale a atenção na Seção 6.3.

4.5. Medição e registro

Os tempos de leitura e validação, de Tarjan e de análise de armadilhas são medidos com `clock()` (tempo de CPU), em milissegundos, e cada execução acrescenta uma linha ao arquivo `resultados/resultados.csv`, com dataset, representação, V , E , número de CFCs, número de armadilhas e os três tempos. Um programa em C sem dependências externas lê o CSV e gera o gráfico em SVG. No modo de matriz, o Tarjan opera sobre uma lista de adjacência derivada da matriz por varredura de V^2 posições; essa derivação é feita antes do cronômetro do Tarjan ser iniciado, portanto não faz parte do tempo medido dessa etapa.

4.6. Ambiente e testes

A compilação usa a imagem Docker `gcc:15` com as opções `-Wall -Wextra -std=c11`, via `docker compose`, o que torna o ambiente reproduzível. Há dois programas de teste: `test_cfc.c` compara lista e matriz aresta a aresta e confere o número de CFCs; `test_armadilha.c` confere os contadores de armadilhas. O módulo `dfs.c` oferece a DFS (ordem de visita e pós-ordem), mas o Tarjan incorpora a sua própria busca e não depende dele.

5. Conjuntos de dados

5.1. Grafo de teste

O grafo `cfc_simple` tem 5 vértices e 6 arestas: o ciclo 0, 1, 2, a aresta (2, 3), a aresta (3, 4) e um auto-laço em 4. Suas CFCs são {0, 1, 2}, {3} e {4}; a única terminal é {4}. Os dois programas de teste passam com esse grafo (3 CFCs; 1 terminal; 1 CFC unitária sem laço; 1 unitária com laço; 1 com mais de um vértice).

5.2. Grafos sintéticos

Cinco grafos (T1 a T5) foram gerados por `gerar_grafo.c` com $E = 4V$. O gerador emite 80% das arestas como um percurso ($u, u+1$ módulo V) que, com probabilidade $1/25$ a cada passo, salta para um vértice aleatório, e os 20% restantes como pares aleatórios. Como o percurso retorna às mesmas regiões, muitas arestas se repetem: em T5, por exemplo, há 17.712 arestas distintas entre as 40.000 geradas (44,3%). Isso explica por que a contagem de duplicadas é alta e por que a matriz, que colapsa repetições, enxerga menos arestas que a lista.

5.3. Malha real do centro de São Paulo

O dado real é um extrato do OpenStreetMap obtido pela Overpass API (base de maio de 2026), cobrindo aproximadamente $3,3 \times 3,4$ km do centro de São Paulo (cerca de 11 km^2 , segundo as coordenadas dos nós). O script `osm_para_grafo.py` mantém as vias de carro (de *motorway* a *living_street*, incluindo as ligações), trata *oneway* igual a *sim* como mão única, *oneway* igual a *-1* como sentido inverso e rotatórias como mão única, e mapeia os identificadores OSM para 0 a $n-1$. O resultado tem 1.291 vias de carro, 6.039 vértices e 8.154 arestas, sem auto-laços nem duplicadas. Vale notar que os vértices são todos os nós das vias, e não apenas as interseções: somente 1.182 nós (19,6%) pertencem a mais de uma via; os demais são nós intermediários que descrevem a geometria. Por isso o grau de saída médio é 1,35, bem abaixo do valor 4 dos grafos sintéticos, e 42,3% das arestas têm aresta reversa (trechos de mão dupla).

6. Resultados e discussão

6.1. Desempenho nos grafos sintéticos

Tabela 1. Resultados nos grafos sintéticos (uma execução por teste; leitura e armadilhas com a lista).

Teste	V	E	CFCs	Term.	Tarjan lista (ms)	Tarjan matriz (ms)	Leitura (ms)	Armad. (ms)
T1	100	400	1	1	0,014	0,011	4,342	0,006
T2	500	2.000	39	23	0,056	0,048	8,054	0,030
T3	1.000	4.000	18	7	0,158	0,158	27,328	0,084
T4	5.000	20.000	114	66	1,130	2,441	433,114	0,828
T5	10.000	40.000	233	129	2,431	10,361	2868,320	5,475

Os números de CFCs e de armadilhas são idênticos nas duas representações, como esperado. Com a lista, o Tarjan cresce de modo compatível com $O(V + E)$: de T4 para T5, V e E dobram e o tempo sobe 2,15 vezes (1,130 para 2,431 ms). Com a matriz, o mesmo salto produz 4,24 vezes (2,441 para 10,361 ms), o que se aproxima de um crescimento quadrático (Figura 2). Os tempos de T1 a T3 ficam abaixo de 0,2 ms e, nessa faixa, a resolução e o ruído de `clock()` dominam; por isso a comparação só é informativa a partir de T4.

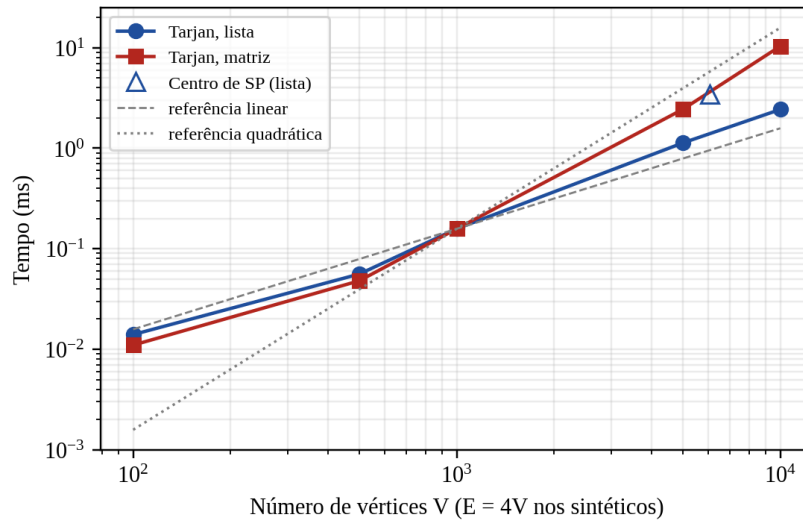


Figura 2. Tempo do Tarjan em função de V , em escala logarítmica, com retas de referência linear e quadrática ancoradas em $T3$.

Há, porém, uma ressalva importante. No código, o cronômetro do Tarjan é iniciado depois da derivação da lista a partir da matriz (Seção 4.5), de modo que a diferença observada não pode ser atribuída à varredura V^2 : ambos os modos executam o mesmo algoritmo sobre listas de tamanho semelhante. Como cada ponto é uma única execução, essa diferença pode refletir variação de execução, ordem das listas ou efeitos de cache, e deve ser confirmada com repetições antes de ser lida como custo da representação. O custo comprovado da matriz neste trabalho é o de memória, tratado a seguir.

6.2. Memória

Tabela 2. Memória estimada das estruturas. Os valores da lista para o centro de SP e para o grafo de teste são os impressos pelo programa; os demais seguem as fórmulas da Seção 4.3.

Conjunto	V	E	Lista (bytes)	Matriz (bytes)	Razão
T5	10.000	40.000	720.016 (0,69 MiB)	400.080.016 (381,5 MiB)	556x
Centro de SP	6.039	8.154	178.792 (0,17 MiB)	145.926.412 (139,2 MiB)	816x
cfc_simples	5	6	152	156	1,03x

Os valores confirmam a vantagem da lista em grafos esparsos: no centro de SP, a matriz ocuparia cerca de 816 vezes mais memória que a lista, e em T5 cerca de 556 vezes. Só em grafos minúsculos e relativamente densos, como o de teste, as duas representações empatam.

6.3. Onde o tempo realmente é gasto

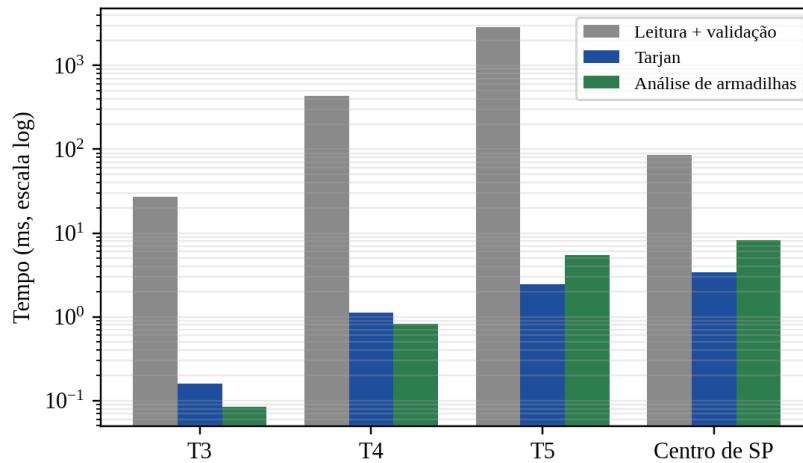


Figura 3. Tempo por etapa (representação em lista), em escala logarítmica.

A Figura 3 mostra que, em T5, a leitura e validação levaram 2.868 ms, cerca de 1.180 vezes o tempo do Tarjan (2,431 ms). A causa está em `io_processar_arestas`, que compara cada aresta com todas as anteriores para contar duplicadas, ou seja, $O(E^2)$: com 40.000 arestas são cerca de 8×10^8 comparações. De T4 para T5, E dobra e o tempo de leitura sobe 6,6 vezes (433 para 2.868 ms), coerente com o comportamento quadrático. Uma tabela de dispersão ou a ordenação das arestas reduziria essa etapa a $O(E)$ ou $O(E \log E)$.

Outro ponto é a análise de armadilhas, que em T5 (5,475 ms) e no centro de SP (8,246 ms) levou mais que o Tarjan, apesar de ser descrita como linear. Isso é compatível com a busca linear do vértice de cada CFC unitária (Algoritmo 1, linha 6): no recorte de SP há 965 CFCs unitárias e 6.039 vértices. Guardar um representante de cada CFC durante a primeira passagem tornaria a etapa $O(V + E)$.

6.4. Malha real: centro de São Paulo

Tabela 3. Resultados no recorte do centro de São Paulo.

Métrica	Valor
Vértices / arestas	6.039 / 8.154
Leitura e validação	85,8 ms
Tarjan / análise de armadilhas	3,4 ms / 8,2 ms
CFCs	979 (965 unitárias e 14 com mais de um vértice)
Maior CFC	4.786 vértices (79,3% da rede)
CFCs terminais (armadilhas)	60: 52 unitárias sem laço e 8 com 2 a 138 vértices
Vértices em CFCs terminais	228 (3,8% da rede)
Terminais com entrada vinda de outra CFC	53 (as outras 7 não recebem arestas externas)
Terminais alcançáveis a partir da CFC gigante	50 de 60

A rede se comporta como esperado para uma região urbana consolidada: uma CFC gigante concentra 79,3% dos vértices, de modo que a maior parte da malha é mutuamente alcançável. As 979 CFCs, porém, devem ser lidas com cuidado: 965 delas são vértices isolados, em geral nós intermediários de trechos de mão única, que formam uma CFC cada por não pertencerem a nenhum ciclo. O número de CFCs, portanto, é inflado pelo grafo não simplificado e informa pouco sobre a estrutura urbana.

Das 60 CFCs terminais, 52 são vértices sem arestas de saída (becos sem saída do grafo) e 8 reúnem de 2 a 138 vértices; a maior delas é a única com porte de “bairro”. Uma análise complementar (`scripts/analise_sp.py`) mostra que 50 das 60 armadilhas são alcançáveis a partir da CFC gigante, ou seja, um motorista da rede principal pode, de fato, entrar nelas; as demais pertencem a outros componentes. A maior parte dos becos sem saída não está na borda do recorte: dos 52 vértices sem saída, apenas 3 ficam a menos de 50 m do limite da área (mediana de 455 m), o que torna improvável que sejam apenas um efeito do corte geográfico. Possíveis causas, não verificadas neste trabalho, são trechos de mão única que terminam em vias excluídas pelo filtro (como vias de pedestres), becos reais e inconsistências de mapeamento.

7. Limitações e ameaças à validade

Medições. Cada tempo é de uma única execução, medido com `clock()`, e vários valores ficam abaixo de 1 ms. O arquivo de resultados acumula linhas de execuções anteriores (caminhos `datasets/` e `dados/` misturados), e as tabelas aqui usam uma linha por teste. O tempo de construção da matriz não é registrado separadamente, e o tempo de leitura também varia entre representações sem motivo algorítmico (em T5, 2.868 ms com lista e 1.908 ms com matriz). Recomenda-se repetir cada teste ao menos 10 vezes e reportar média e desvio.

Dados sintéticos. Os grafos gerados têm muitas arestas repetidas (apenas 44% distintas em T5) e muitas CFCs, o que os torna úteis para exercitar o algoritmo, mas não representativos de uma cidade. O grau médio de 4 também não corresponde ao dado real (1,35).

Dado real. O grafo não é simplificado (vértices incluem nós de geometria), cobre uma única região de cerca de 11 km², e não foi validado contra outra fonte. A definição de armadilha (CFC terminal) é conservadora e não identifica, por exemplo, regiões que só podem ser deixadas por rotas muito longas.

Implementação. O Tarjan é recursivo; em malhas com centenas de milhares de vértices, a profundidade da recursão pode estourar a pilha de execução, e uma versão iterativa seria mais segura. A análise de alcançabilidade e de distância à borda citada na Seção 6.4 não faz parte do programa em C: foi feita pelo script complementar `scripts/analise_sp.py`.

8. Conclusão

A Fase I entregou um pipeline completo, do dado cru do OpenStreetMap ao registro de resultados, e uma definição operacional de armadilha de trânsito como CFC terminal. Os experimentos confirmam a vantagem da lista de adjacência para malhas esparsas, principalmente em memória (cerca de 816 vezes menos que a matriz no centro de SP), e mostram que o Tarjan processa 6.039 vértices em poucos milissegundos. No recorte do centro de São Paulo, a rede é dominada por uma CFC gigante (79,3%), e foram identificadas 60 CFCs terminais, das quais 50 são alcançáveis a partir da região principal.

A análise também revelou melhorias claras: substituir a contagem quadrática de duplicadas, guardar um representante por CFC, repetir as medições com estatística e simplificar o grafo real contraindo nós intermediários. Para a Fase II, planeja-se incluir pesos (tempo de trânsito), analisar rotas e otimização sobre o mesmo grafo, e investigar caso a caso as armadilhas encontradas, por exemplo sobrepondo-as ao mapa.

Referências

- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139.
- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. (2009). *Introduction to Algorithms*. 3. ed. MIT Press.
- ISO/IEC (2011). *ISO/IEC 9899:2011 — Programming languages — C*.
- OpenStreetMap contributors. Dados de mapa disponíveis sob a licença ODbL em <https://www.openstreetmap.org>.
- Sharir, M. (1981). A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1), 67–72.
- Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2), 146–160.